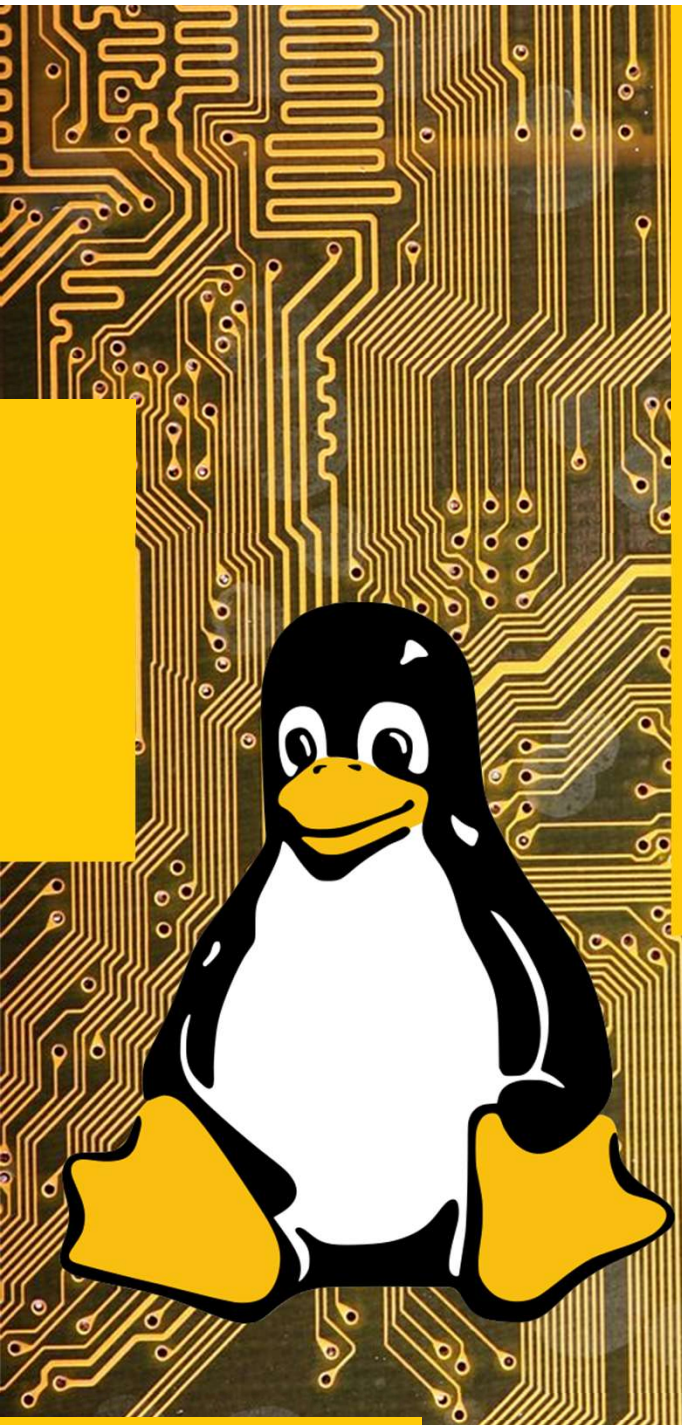


INHA UNIVERSITY IN TASHKENT

SOC 3010 OPERATING SYSTEMS

IPC III

Sockets and Concurrent Servers

INSTRUCTOR: Jalolliddin Yusupov

WEEK IV Course Topics & Weekly Lecture Breakdown

Lecture Topic: Sockets and Concurrent Servers (TLPI Ch 56-61)

- The socket system calls, the listen queue, and message framing over a byte stream. Handling the death of a peer and the reuse of an address. Unix domain sockets, peer credentials, and passing an open file between processes.
  Server architectures: one client at a time, a process per client, a pool of processes; select and poll.

Until today, every two processes we connected lived on the same machine.

Today the pipe gets long enough to reach another computer.

Reading

Silberschatz §3.8 (communication in client–server systems).
Optional: TLPI ch. 56–60.

The Last Branch of the IPC Tree

mechanism	who can talk	where	what travels
pipe (W3)	a parent and its children	same machine	one way, bytes
FIFO (W3)	any process that knows the path	same machine	one way, bytes
message queue (W4)	any process that knows the name	same machine	whole messages
shared memory (W4)	processes that share it	same machine	no messages at all: the same bytes
socket (today)	**any process that knows the address**	**any machine in the world**	**two ways, bytes**

A new job for the OS

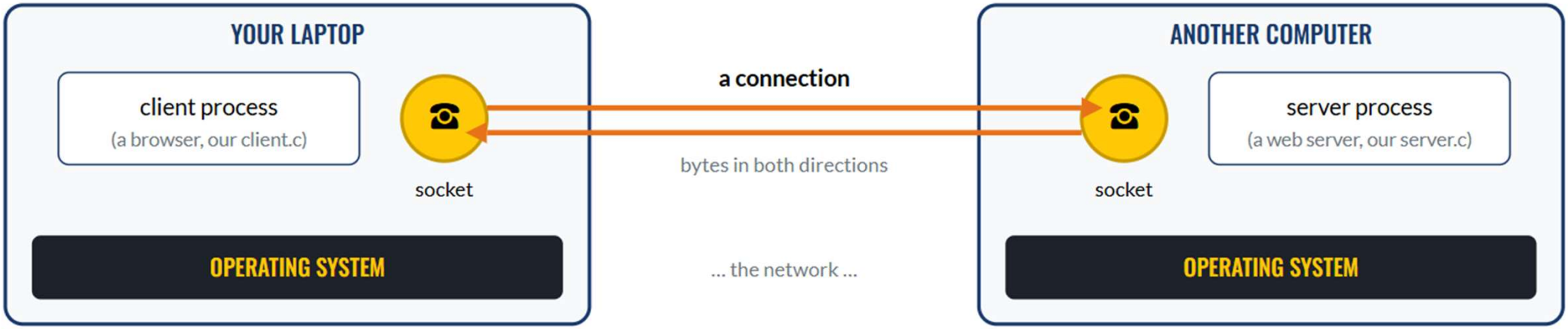
Until now the OS moved data between two of **its own** processes. With a socket, **two operating systems** cooperate: yours hands the bytes to the network, the other one delivers them to the right process.

Why it matters

Every web page, every chat message, every online game, every database query and every Telegram sticker you ever sent went through a socket.

A socket is a pipe that can leave the machine — and works in both directions.

What Is a Socket?



Four names, all new today

Socket — one **end** of a conversation between two processes.
The OS creates it and gives it to you as a **file descriptor** (the "ticket number" of Week 3).
Connection — two sockets joined together: a two-way stream of bytes.
Server — the process that **waits** to be called.
Client — the process that **calls**.

Think of a telephone

socket = the **phone** on your desk
connection = the **call** · server = a shop waiting by its phone
client = the customer who dials.
Both sides can talk and listen on the same call — a pipe could only do one direction.

A socket is a telephone the OS hands to your process.

The Address: Which Machine, Which Process?

IP address	**which machine** — like the address of a building. `192.168.1.7` , `142.250.74.46`
port	**which process on that machine** — like the apartment number. A number from 0 to 65535.
`localhost`	a special name: **this same machine** (`127.0.0.1`). Client and server on one laptop — perfect for testing.

port	who usually lives there
22	remote login (ssh)
80 / 443	web servers (http / https)
5432	PostgreSQL database
5000	**our examples** (below 1024 only the administrator may use)

The IP address finds the machine; the port finds the process.

The OS keeps the table

port → process.
When bytes arrive for port 5000, the OS finds the process that owns port 5000 and wakes it up.
Two processes cannot own the same port: the second one is told Address already in use.



One word: TCP

The kind of connection we use: **reliable** and **in order** — nothing lost, nothing swapped. How it manages that is the Networks course.

A Phone Call in Pseudo-Code

SERVER — THE SHOP

```
Server:
  L = listen(5000)      /* install a phone,
                           number 5000     */
  C = accept(L)         /* wait for a ring,
                           pick up         */
  name = read(C)        /* listen          */
  write(C, "hello " + name)
                        /* answer          */
  close(C)              /* hang up         */
```

CLIENT — THE CUSTOMER

```
Client:
  C = connect("shop-address", 5000)
                        /* dial            */

  write(C, "Ali")       /* talk            */
  reply = read(C)       /* listen          */
  print(reply)          /* hello Ali       */

  close(C)              /* hang up         */
```

Only three new verbs

listen, **accept**, **connect**. After that it is read, write, close- the same three calls you used on files, pipes and FIFOs. To your program a connection is just another file descriptor.

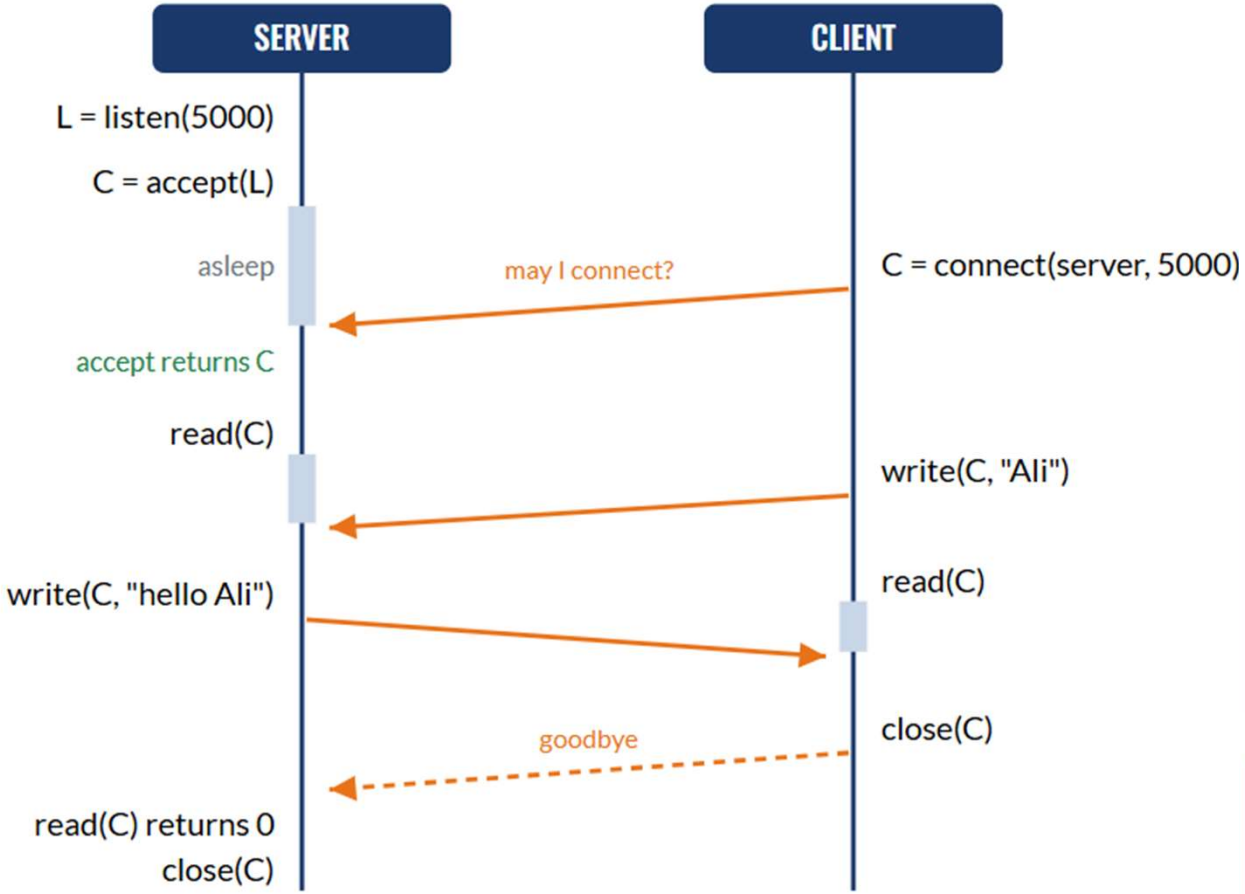
Who must start first?

The **server**. A client that dials a number where nobody is listening gets Connection refused at once.

Server: listen, accept. Client: connect. Then both simply read and write.



Draw It: Who Sleeps When?



Time goes down the page

Each vertical line is one process. An arrow is something travelling through the network. A grey block means **the process is asleep.**

Two calls that sleep

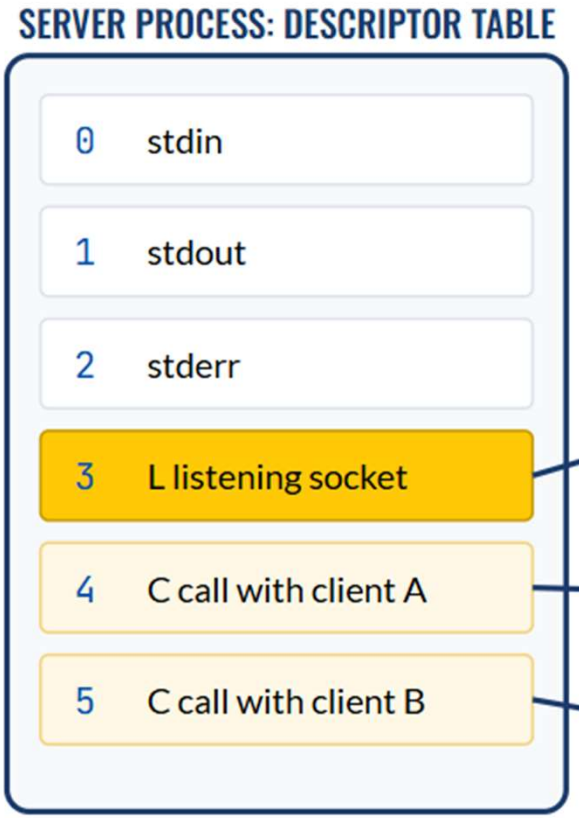
`accept` sleeps until a client connects. `read` sleeps until bytes arrive. The OS wakes the process — a sleeper uses **no CPU** (state `S` in `ps`), exactly like a reader on an empty pipe.

The end of a call

When one side closes, the other side's `read` returns **0** — end of file, the same signal as a pipe whose writers are all gone (Week 3).

accept sleeps for a caller; read sleeps for data; a closed call reads as 0.

Two Kinds of Socket in a Server



The listening socket

It never carries a single byte of conversation. Its only job: receive **new calls**. A shop has **one** public number.

the reception phone
port 5000

```
L  = listen(5000)      /* descriptor 3 */
C1 = accept(L)         /* descriptor 4 */
C2 = accept(L)         /* descriptor 5 */
```

private line to client A

private line to client B

The connection socket

Every `accept` returns a **new** descriptor: a private line to exactly **one** client. Talk on it, then `close` it. Two clients = two private lines, and their bytes can never mix.

"Lowest free number"

The OS hands out descriptors the same way as in Week 3: 0, 1, 2 are taken, so the first socket is 3.

One listening socket; one new connection socket for every client.

What the OS Really Does Behind the Three Verbs

our verb	real system calls	what the OS does

`listen(5000)`	`socket()`
	`bind()`
	`listen()`
		creates the phone (a new descriptor)
		writes "port 5000 → this process" into its table
		switches the phone on and starts a **waiting queue** for callers

`accept(L)`	`accept()`	takes the first caller out of the waiting queue and gives you a **new** descriptor for that call; sleeps if the queue is empty
`connect(host, 5000)`	`socket()` `connect()`	creates the client's phone your OS and the server's OS exchange three short messages ("hello?" — "hello, I hear you" — "good"): the call exists

The listen queue: callers on hold

While your server is busy with one client, **the OS answers the phone for you** and keeps the new callers in a queue. They are connected, but nobody is talking to them yet.

Do I have to memorise five calls?

No. In this course the five calls and their structures are packed into a helper, `net.h` — the same idea as `shared_mem.h`. You should **recognise** the names when you meet them in real code.

You say three verbs; the OS makes five system calls and keeps the queue.

net.h: Networking in Three Calls, Given to You

NET.H — ON ECLASS; READ IT ONCE, NEVER MEMORISE IT

```c
#include "net.h"

int server_listen(int port);              /* returns L */
int server_accept(int L);                 /* returns c */
int client_connect(const char *host, int port);
                                          /* returns c */

int read_line(int c, char *buf, int size);
              /* one whole line = one message */
```

Each call returns a **file descriptor**. From then on: `read(c, …)`, `write(c, …)`, `close(c)` — nothing new.

How to use it

Put `net.h` in the same folder as your `.c` file, write `#include "net.h"`, and compile as always: `<p>` `gcc -o server server.c`.

If something goes wrong

The helpers print the reason and stop the program: `bind: Address already in use` — another server already owns that port. `connect: Connection refused` — no server is listening there.

Why a helper?

The real calls need three structures and about 25 lines of setup that are the same in every program on Earth. The **logic** is what we study.

Three calls to get a descriptor; after that it is read, write, close.

Example: The Echo Server

```c
/* echo_server.c — sends back whatever a client says. One client at a time. */
#include <stdio.h>
#include <unistd.h>
#include "net.h"

int main(void) {
    int L = server_listen(5000);            /* the reception phone */
    printf("server: waiting on port 5000\n");

    while (1) {
        int c = server_accept(L);           /* sleeps until a client calls */
        printf("server: a client connected\n");

        char buf[100];
        ssize_t n;
        while ((n = read(c, buf, sizeof buf)) > 0)   /* 0 = client hung up */
            write(c, buf, n);                        /* send it back       */

        printf("server: the client left\n");
        close(c);                           /* hang up this call */
    }
}
```

Read it top to bottom

Open port 5000 · forever: take the next caller, send back everything they say, hang up when they leave.

The inner loop

`read` returns how many bytes arrived; **0** means the client closed. Until then, whatever came in goes straight back out.

"Echo"

The "Hello, World" of servers: it has no logic of its own, so everything you see is plumbing.

Example: The Client

```c
/* client.c — sends one message to the server and prints the answer.
   Usage: ./client <host> <message>        e.g.  ./client localhost hello */
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include "net.h"

int main(int argc, char *argv[]) {
    if (argc != 3) { printf("usage: %s <host> <message>\n", argv[0]); return 1; }

    int c = client_connect(argv[1], 5000);      /* dial the number */

    write(c, argv[2], strlen(argv[2]));         /* say something   */
    write(c, "\n", 1);                          /* end of message  */

    char reply[100];
    read_line(c, reply, sizeof reply);          /* listen          */
    printf("client: the server said \"%s\"\n", reply);

    close(c);                                   /* hang up         */
    return 0;
}
```

Read it top to bottom

Dial `<host>` on port 5000 · send the message and a newline · read one line back · print it · hang up.

`argv[1]`, `argv[2]`

The words you type after the program name: `./client localhost hello` gives `argv[1] = "localhost"`, `argv[2] = "hello"`.

Why the `"\n"`?

It marks the **end of the message**, so `read_line` on the other side knows where to stop. More on slide 16.

Example: The Client

Start the server first; then anyone who knows the address and the port can call.

TERMINAL 1 — THE SERVER

```
$ gcc -Wall -o echo_server echo_server.c
$ ./echo_server
server: waiting on port 5000
server: a client connected
server: the client left
server: a client connected
server: the client left
```

TERMINAL 2 — THE CLIENT

```
$ gcc -Wall -o client client.c
$ ./client localhost hello
client: the server said "hello"
$ ./client localhost "operating systems"
client: the server said "operating systems"
```

Now leave the machine

On the same Wi-Fi, ask a classmate for their address (`hostname -I`) and run `./client 192.168.1.7 hello` . Their terminal prints "a client connected". You have just built the internet's smallest service.

Two errors you will meet today

`connect: Connection refused` — you started the client first.
`bind: Address already in use` — you started the server twice.

Exercise: A Socket Is a Stream of Bytes

```
Client:
  C = connect(server, 5000)
  write(C, "HELLO")
  write(C, "WORLD")
  close(C)

Server:
  L = listen(5000)
  C = accept(L)
  n = read(C, buf, 100)
  print(buf)
```

Which outputs are possible?

(1) `HELLOWORLD`

(2) `HELLO`

(3) `HEL`

(4) `WORLDHELLO`

Hint

Remember Week 3: what did a pipe do with three writes of `AB` ?

Answer : Framing the messages

Framing: marking where a message ends

Framing — the rule both sides follow to cut the stream back into messages. Two classic rules:

1. every message ends with a special byte, usually `'\n'`;
2. every message starts with its length.

What we use

Rule 1. The client sends `"hello\n"`, and `read_line()` from `net.h` keeps reading until it sees the `'\n'`. One line = one message.

In real life

HTTP uses both: lines for the headers, a length for the body.

The lesson

A socket remembers the **bytes and their order**, not your `write` calls. One write is **not** one message — exactly like a pipe, and unlike a message queue.

One write is not one message: agree on where a message ends.

```
Client:
  C = connect(server, 5000)
  write(C, "HELLO")
  write(C, "WORLD")
  close(C)

Server:
  L = listen(5000)
  C = accept(L)
  n = read(C, buf, 100)
  print(buf)
```

HELLOWORLD	**possible** — both writes arrived before the read
HELLO	**possible** — the read ran between the two writes
HEL	**possible** — the network may deliver a message in pieces
WORLDHELLO	**impossible** — TCP never changes the order

Unix Domain Sockets: The Same Phone, Inside One Machine

A socket whose address is a **file name** instead of IP + port. It can only reach processes on the **same machine**.

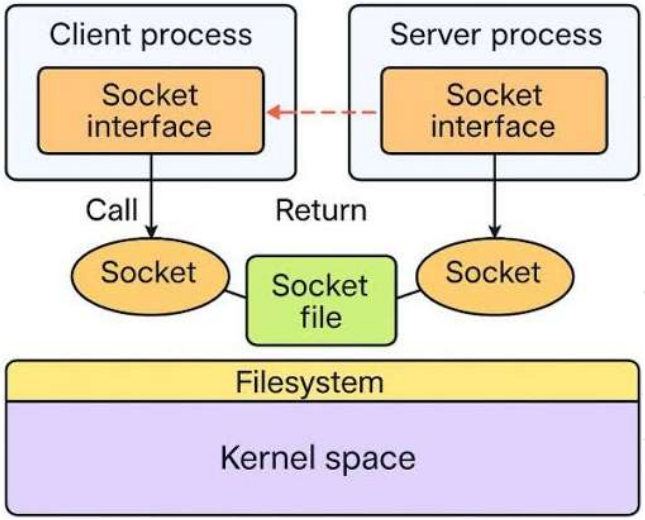
```
Server:                      Client:
  L = listen("/tmp/shop")      C = connect("/tmp/shop")
  C = accept(L)                write(C, ...)
  ...                          ...
```

```
$ ls -l /tmp/shop
srwxr-xr-x 1 root root 0 Oct  1 09:39 /tmp/shop
```

The first letter is the file type: `s` = socket (a FIFO showed `p`, a directory `d`).

Who uses it

Docker (every docker command talks to /run/docker.sock), **PostgreSQL** and **MySQL** for local clients, your **desktop** (every window talks to the display server).



same verbs	listen, accept, connect, read, write, close
no network	the bytes never leave the OS: faster than localhost
file permissions	the OS decides who may call, exactly as for a file
vs a FIFO	two-way, and **every client gets its own private line** — in a FIFO all writers are mixed into one stream

Same calls; the address is a file name, and the call never leaves the machine.

Homework

1. Run it. Compile `echo_server.c` and `client.c` (with `net.h`, on eClass). Run them in two terminals. Then connect to a classmate's server over Wi-Fi. Screenshot both terminals.

2. Change it. Make the server answer in **CAPITAL LETTERS** (`toupper` from `<ctype.h>`): the client sends `hello`, the server replies `HELLO`.

3. On paper. Two clients, A then B, each send one message to the echo server. Draw the time diagram (three vertical lines) and the server's descriptor table at the moment it is talking to B.

Think about this before next lecture

Client A connects and then goes for lunch without sending anything. Client B connects one second later. What happens to B — and for how long?



Questions?